

Exploring Activation Functions to Improve Gradient Flow and Stability in Neural Networks

Maurice Murillo

Computer Science Senior Seminar

November 17, 2025

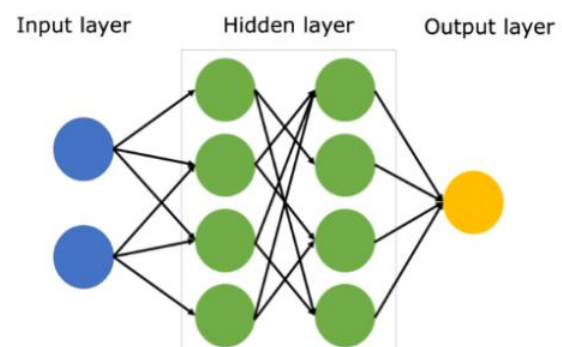
0. Abstract

This paper investigates whether a custom activation function can improve gradient flow and training stability in a neural network compared to common functions such as Tanh and ReLU. After reviewing some of the core concepts such as gradient descent, backpropagation, and loss functions, I compare these activation functions against my custom function on a supervised learning task, using a neural network implemented from scratch in Python using the Wine Quality dataset from the UC Irvine Machine Learning Repository [2]. The goal is to create a binary classifier to predict good wine or bad wine. The performance of the activation functions is then compared on several criteria including accuracy, convergence speed, and gradient behavior.

1. Introduction

Neural Networks are computational models inspired by the neurons in the human brain. A neural network is essentially a list of layers. They consist of interconnected nodes (neurons), and each node applies a simple mathematical transformation and passes the result forward to the next layer. The network then makes a prediction and calculates the error using a loss function. The error is then propagated backwards, using gradient descent to update the parameters of the mathematical transformation of the nodes [1, 5]. The goal is to minimize the error over time by adjusting the network's weights and biases so that its predictions become increasingly accurate. The role of a neural network is to construct a function, or a relationship between the inputs and the correct outputs. Activation functions are used in neural networks to introduce non-linearity into the model. Non-linearity determines how signals flow forward and how gradients flow backwards during training. The activation function decides whether a neuron should be activated and helps the model make complex decisions and predictions. Without this function, a neural network would behave like a linear regression model no matter how many layers it has. Unfortunately, there are problems that can occur with different activation functions. We ask whether a custom activation function improves gradient flow and stability in a neural network compared to commonly used activation functions.

Neural Network Layout



2. Background

2.1 Neural Networks

A neural network is a computational model designed to learn from data and make accurate predictions. It is organized into three layers: input layer, hidden layer(s), and output layer. The input layer is where the data is introduced in the model. The hidden layer is where the learning occurs; the model tries to learn a function or mapping that connects the input features to the correct output. Then, the output layer produces the model's prediction [1, 5]. Neurons are fed sequentially from one layer to the next. A neuron computes:

$$z = \Sigma(w * x) + b$$

$$a = f(z)$$

Where x are the inputs, w are weights, which determine the influence the input has on the output; b is the bias, which allows the node to shift the function up or down. And f is the activation function applied to the z weighted sum. This computation is applied to every node in every layer of the neural network. The outputs are then fed sequentially into the next layer until we reach the final output value.

2.2 Forward Pass

A neural network begins by computing the forward pass. This is the process of passing the input vector through the layers of the neural network to produce an output. Each neuron performs a mathematical transformation, and although it's simple for a single neuron, its complexity grows as layers are stacked sequentially [5]. The forward pass is an essential part of the training process because it makes predictions of the model. This prediction is used to measure the error and update the parameters to make a better prediction in the next forward pass.

2.3 Loss Function

The loss function determines how wrong the model prediction was compared to the true values. Without the loss function, the network would have no way of knowing if its predictions are improving or how its parameters should be adjusted. To calculate it, we typically use the **squared error loss**, which is one of the simplest ways of calculating the loss in a neural network. The squared error for a single prediction is represented by:

$$L = \frac{1}{2}(y - y')^2,$$

Where y is the true label and y' is the network's prediction (a Boolean value). We use $\frac{1}{2}$ and square the difference is to help in our implementation of the backpropagation algorithm. We need the derivatives for backpropagation, and formatting the equation this way makes it easier for us to see clearly how each activation function affects the derivatives that propagate backward through the network [5, 7]. We want to minimize the loss function by applying the best activation function to the neuron before the output.

2.4 Backpropagation

After calculating the loss function, we know the errors in the network. But knowing the error isn't enough, we have to find out how to change the parameters to make the predictions more accurate. Backpropagation is a gradient computation method used for training neural networks. It computes how much each weight contributed to the total error (or loss) [5]. The result is a set of gradients (partial derivatives) of each weight. These gradients are used to update the weights so the network can make accurate predictions. It is an efficient application of the chain rule. It starts by iterating backward from the output layer and computes how each output neuron contributed to the total loss. Then, it continues layer by layer toward the input, distributing the error to those neurons [1]. This allows the weight from each neuron to receive a "blame" signal that tells them how it should be updated to predict better.

To update our weight w , we need to compute the derivative $\frac{\partial L}{\partial w}$. Backpropagation does this by breaking the expression into smaller parts using the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$\frac{\partial L}{\partial a}$ measures how the prediction affects the loss. $\frac{\partial a}{\partial z}$ is the derivative of the activation function. $\frac{\partial z}{\partial w}$ shows how the weights affect the neurons pre-activation output. Multiplying these three quantities gives the weight gradients. We do this for every neuron in every layer of the neural network. Every weight in the network receives a gradient, and gradient descent uses those gradients to update the parameters.

2.5 Gradient Descent

Backpropagation calculates the gradients, which compute how each weight needs to change to make the predictions more accurate. Gradient descent is the optimization algorithm used to update the neural networks parameters (weights and biases) so the predictions can get better over time [5]. It uses the gradients to update the weights by moving them in the direction that reduces the error. Mathematically, gradient descent moves down the surface of the loss function. We keep going down until we reach a minimum. This minimum becomes the new optimized parameter for the model at the current weight. Once the gradient of the loss with respect to weight w is known, the update rule is:

$$w_{new} = w_{old} - \eta \left(\frac{\partial L}{\partial w} \right).$$

This rule is called the Gradient Step, where η is the learning rate. The learning rate controls how big of a step you take down the loss surface. This update slightly changes the weight in the direction that lowers the loss, effectively making the predictions more accurate.

2.6 Activation Functions

An activation function is a mathematical function applied to each neuron's weighted sum. It decides what information passes forward, how the gradients flow backwards during backpropagation, and how well the network learns. It decides whether the neurons activate and how strong they are. Activation functions make deep learning possible. Early activation functions such as Hyperbolic Tangent (Tanh) and Sigmoid were used because of their smooth, differentiable curves. These functions help the model converge faster and allow gradients to propagate more smoothly through the network. The Tanh equation is represented by:

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Tanh squashes large positive/negative inputs in the range (-1, 1), making it a zero-centered output. This allows weights and biases to update evenly during training. It also improves the gradient flow by maintaining a

steeper gradient around its center, which helps make learning easier during backpropagation. For large input values, the output saturates near -1 or $+1$. This means that its gradient (derivative) approaches 0, because of the large input values. This also makes weight updates almost zero because the gradients are very small. This is the *vanishing gradients problem*, which can slow or stop the learning process [6, 7]. Since the equation uses exponential operations to compute, there is also a higher computational cost associated with this function.

The Rectified Linear Unit (ReLU) function is a much simpler and more cost-effective model compared to Tanh. ReLU's equation is defined as

$$ReLU(x) = \max(0, x)$$

This activation function is simple. If the input is non-negative, we pass it unchanged, else we pass zero. This allows the positive inputs to have larger gradients, which is faster and provides more stable training compared to Tanh because it does not saturate its positive region. In addition, ReLU avoids the vanishing gradient problem that Tanh experiences. The derivatives are 1 for all positive values and 0 otherwise, so gradients won't shrink during backpropagation. Since we don't worry about vanishing gradients, we can train deep learning models with more layers and solve more complex problems. Unfortunately, there are also issues with the ReLU activation function. One main issue is that we basically ignore the negative values and set them to zero. Consequently, ReLU will cause extreme information loss in a network [6]. This is the *dying neurons problem*. If neurons input always produce a negative value, its output and gradient become zero. That neuron stops learning completely, so it is effectively "dead" and useless for the rest of the training. Another issue is the *exploding gradients problem*. During the forward pass, positive activations can grow larger as they pass through the layers. This creates large values in the network, but during backpropagation we compute the gradients using the chain rule. This involves multiplying derivatives across layers, so if any of the values are large, the gradients will be too. As the large gradients propagate backwards, repeated multiplication can cause them to grow exponentially, leading to instability during training.

3. Problem

Although Tanh and ReLU are commonly used activation functions in neural networks, they introduce problems that can affect training performance. Tanh is smooth and produces zero-centered activations, but it causes vanishing gradients and can be expensive to compute. ReLU is simple, efficient, and fast, but it causes dying

neurons and exploding gradients. Given these issues, we explore whether it is possible to design a custom activation function that reduces the weaknesses of Tanh and ReLU while still maintaining stability and gradient flow during training?

4. Methods

4.1 Datasets

The experiments will be using the Wine Quality dataset from the UC Irvine Repository [2]. The model is trained to perform binary classification, where wines with a quality score of 6 or higher are labeled as 1 (good), and all others are labeled as 0 (bad). During the cleaning process, the dataset is standardized so that all features are on the same scale [5]. This will be important to improve training stability. Then, data is then split into training and testing sets and ready for training. This ensures that the model is evaluated on unseen data after training. The code demonstrates how the dataset is loaded, cleaned, standardized, and split for training. This dataset will be used through all the experiments to test the performance of Tanh, ReLU and the custom activation function.

Data Cleaning and Train Splitting:

```
df = pd.read_csv("winequality-red.csv", sep=";")
X_raw = df.drop('quality', axis=1).values
y_raw = (df['quality'] >= 6).astype(int).values
X = StandardScaler().fit_transform(X_raw)
X_train, X_test, y_train_raw, y_test_raw = train_test_split(X, y_raw, test_size=0.2, random_state=42)
```

4.2 Neural Network Architecture

To evaluate the activation functions, I created a neural network from scratch in Python. This architecture is used specifically for training on the wine dataset. It includes 11 input features because there are 11 chemical properties in this dataset, 10 hidden layers with 10 neurons, and 1 output neuron for binary classification. The output will be either 0 or 1 depending on whether it thinks the wine is good or bad. The code shows how the wine-quality network is constructed. There are hidden layers and the output layer components. This structure ensures that each neuron is fully connected to the previous layer, and it allows easy switching between activation functions.

Neural Network Architecture

```
NUM_INPUTS = X_train.shape[1]
NUM_HIDDEN = 10
NUM_OUTPUTS = 1
def init_network():
    hidden_layer = [[random.uniform(-1, 1) for _ in range(NUM_INPUTS + 1)] for _ in range(NUM_HIDDEN)]
    output_layer = [[random.uniform(-1, 1) for _ in range(NUM_HIDDEN + 1)] for _ in range(NUM_OUTPUTS)]
    return [hidden_layer, output_layer]
```

The forward pass computes the network's predictions. As shown in the code, each neuron computes a weighted sum of the inputs, adds a bias, and applies an activation function. All the activated outputs are saved in a vector, and the network's final prediction is also saved.

Feed Forward Function

```
def feed_forward(network, input_vector):
    outputs = []
    for layer in network:
        input_with_bias = input_vector + [1]
        output = [activation(neuron, input_with_bias) for neuron in layer]
        outputs.append(output)
        input_vector = output
    return outputs
```

4.4 Backpropagation Implementation

Once the forward pass computes the prediction, the next step is to compute the gradients. Gradients tell us how each parameter contributed to the overall error. This is done during backpropagation, which applies the chain rule across each layer. The function shown below calculates how much each weight in the network should change. The deltas are how much that neuron contributed to the overall output error. We use the deltas to calculate the gradients, which tell us how much that neuron should change to reduce the error. The function returns a full set of gradients for every weight in the network. These gradients are then passed into the gradient descent step and updated.

Squared Error Gradients Function (Backpropagation)

```
def square_error_gradients(network, input_vector, target_vector):
    hidden_outputs, outputs = feed_forward(network, input_vector)
    output_deltas = [(derivative_of_activation) * (output - target)
                     for output, target in zip(outputs, target_vector)]
    output_grads = [[output_deltas[i] * hidden_output for hidden_output in hidden_outputs + [1]]
                   for i, _ in enumerate(network[-1])]
    hidden_deltas = [hidden_output * (1 - hidden_output) * dot(output_deltas, [n[i] for n in network[-1]])
                    for i, hidden_output in enumerate(hidden_outputs)]
    hidden_grads = [[hidden_deltas[i] * inp for inp in input_vector + [1]]
                   for i, _ in enumerate(network[0])]
    return [hidden_grads, output_grads]
```

4.5 Gradient Descent

Gradient descent is used to update the network parameters, so the loss gradually decreases over time. This is the process that the neural network uses to “learn”. This algorithm moves the weights in the direction that decreases the loss the fastest. The code shows the implementation of the gradient step. This is what does the actual update to the parameters in the network. The step size is controlled by the learning rate, which determines how big a change (step) we want to take when updating. Normally, this is a small value, as we want to improve the model little by little with each iteration. It uses the gradients we calculated during backpropagation and multiplies it by the learning rate. We subtract the scaled gradient from the old weight, which moves the parameter in the direction that reduces the loss. Repeating this process for every parameter is what allows the model to improve over time.

Helper Functions for Gradient Descent

```
def dot_product(inputs, weights):
    return sum(inp * weight for inp, weight in zip(inputs, weights))

def vector_add(gradients, steps):
    return [gradient + step for gradient, step in zip(gradients, steps)]

def scalar_multiply(scalar, vector):
    return [scalar * v for v in vector]

def gradient_step(weights, gradient, step_size):
    step = scalar_multiply(step_size, gradient)
    return vector_add(weights, step)
```

5. Evaluation Criteria

5.1 Training Accuracy

To determine whether the custom activation function performed better than Tanh and ReLU, we can check the training accuracy. The training accuracy measures how well the model fits with data it has already seen. Activation functions with stable gradient flow normally achieve higher training accuracy because the network can adjust its weights more effectively. By comparing the training accuracies in the experiments, I can see which activation function trains the model to learn patterns more efficiently. We train the network by repeatedly performing forward passes, computing gradients using the loss, and applying gradient descent to update the parameters. We first initialize the learning rate and the epochs. An epoch is one full pass through the training data: it's an iteration. We do the forward pass in the *square_error_gradients()* function, calculate the gradients, and update the parameters in the neural network using the gradient step. There is a set of gradients for each layer in the network.

Evaluating Training and Testing Data:

```
def evaluate_on_data(network, X_data, y_data, func):
    correct = 0
    for x, y in zip(X_data, y_data):
        _, activations = feed_forward(network, x.tolist(), func)
        pred = 1 if activations[-1][0] >= 0.5 else 0
        if pred == int(y[0]):
            correct += 1
    return correct / len(X_data)
```

5.2 Testing Accuracy

The testing accuracy checks to see how well the model generalizes to unseen data. This metric is more important than training accuracy because high training accuracy doesn't necessarily mean the model will behave the same when introduced with new information. I will train the models under the same conditions (learning rates, epochs, dataset), and I will be able to see which activation function learns patterns the best. It has a similar setup as the training accuracy function, but it is training the model using the testing data.

5.3 Convergence Speed

Convergence speed is how quickly the model reaches a good accuracy or low loss (low error). An activation function with smoother derivatives will reach better predictions faster. To measure the convergence speed in a consistent and meaningful way, I will define a performance threshold. For example, 80% training accuracy threshold. I will record the epoch at which each activation function reaches that threshold. The activation function that reaches the threshold in the fewest epochs demonstrates better gradient flow and more efficient learning.

5.4 Loss Curve Behavior

The loss curve will show whether the activation function was stable. Stable functions often produce smooth, steadily decreasing loss curves. Unstable activation functions display plateaus (where the loss doesn't go down and accuracy doesn't go up) or sudden spikes. By plotting the loss curve, I will visually be able to see whether learning is stable, whether the model gets stuck, and how quickly it reduces loss.

6. Custom Activation Function

For the custom activation function, I want to bring in the best parts of Tanh. I think using Tanh is important because it gives a zero-centered output, which helps the network learn faster and keeps gradients flowing smoothly during backpropagation. The custom activation function is:

$$custom(x) = 0.1x + Tanh(x).$$

This function keeps the output centered around zero, but it also fixes one of Tanh's biggest weaknesses. When input becomes very large or very small, Tanh saturates at +1 or -1, which makes the derivatives extremely small. This slows down the learning process and makes training unstable. By adding the small linear term $0.1x$, the function never fully flattens out, so the gradients never become zero. The function keeps the negative values (avoids dying neurons), keeps the output derivatives away from zero (avoids vanishing gradients), and keeps gradient flow stable during training (reduces changes of exploding gradients). Overall, I think this function will provide a good balance between non-linearity and strong gradients in the network.

7. Analysis

7.1 Overall Performance Comparison

To compare the activation functions fairly, all models were trained using the same neural network architecture, dataset split, and optimization procedure. The performance was evaluated primarily using **testing accuracy and final test loss**, since these metrics best reflect generalization on unseen data. Table 1 summarizes the best-performing results for each activation function across all tested learning rates and epoch counts.

Table 1: Best Results by Activation Function

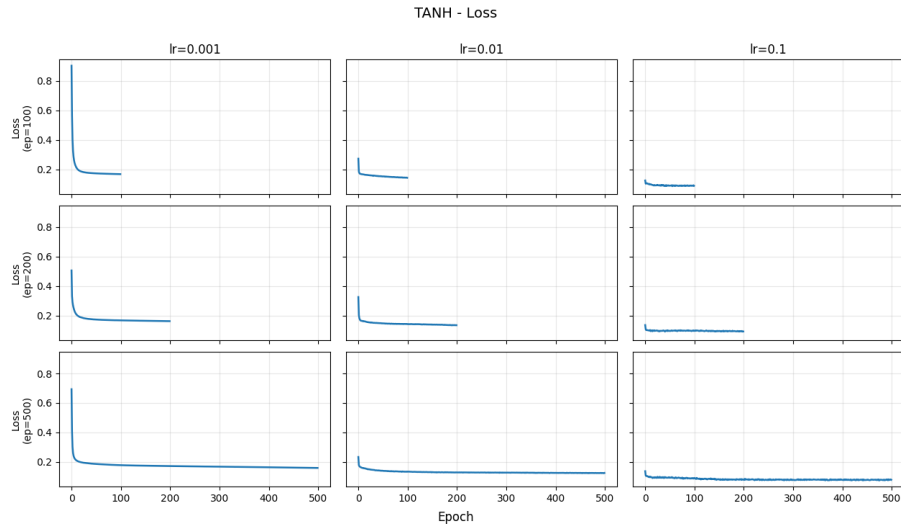
Activation Function	Learning Rate	Epochs	Train Accuracy	Test Accuracy	Final Test Loss
Tanh	0.001	100	74.75%	76.88%	16.81%
ReLU	0.01	500	81.08%	77.19%	17.64%
Custom	0.001	500	77.48%	78.88%	16.74%

The custom activation function achieved the highest testing accuracy and lowest final test loss among all activation functions tested. For this experiment, we can say that it performed better than the other two activation functions.

7.2 Tanh Performance Analysis

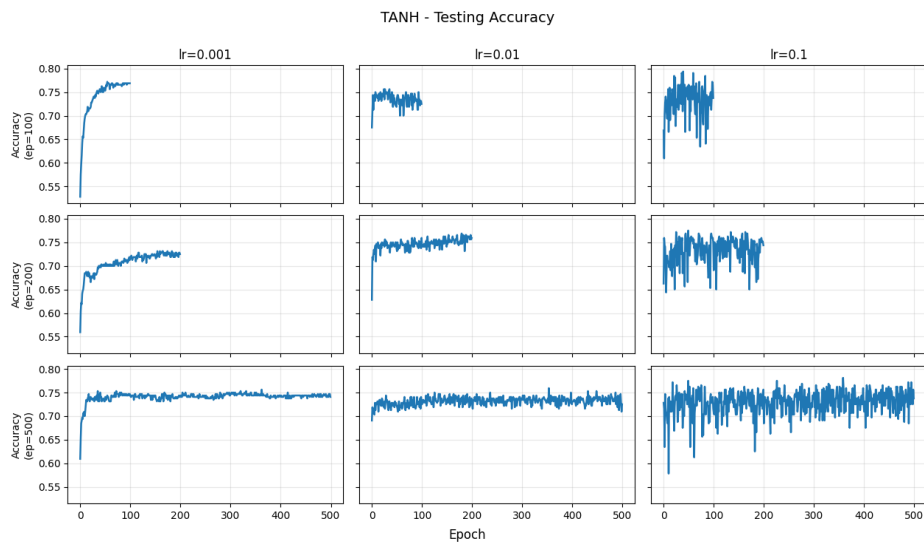
Tanh results demonstrated stable but limited performance across learning rates. At a low learning rate (0.001), the model showed smooth loss reduction and early generalization and achieved its highest test accuracy at 100 epochs. As shown in Figure 1, the loss decreases rapidly during the early stages of training and then transitions into a gradual decline as the number of epochs increases. This behavior indicates stable optimization initially, followed by diminishing improvement as the model approaches a plateau.

Figure 1: Tanh Loss Curves



The testing accuracy improves quickly early in the training and then fluctuates across later epochs, as shown in Figure 2. Additional training beyond 100 epochs does not produce meaningful results and, in some cases, slightly reduces the testing accuracy. The loss and testing accuracy graphs indicate that while Tanh provides smooth and stable learning, its performance is constrained by vanishing gradients that limit long-term improvement with more epochs.

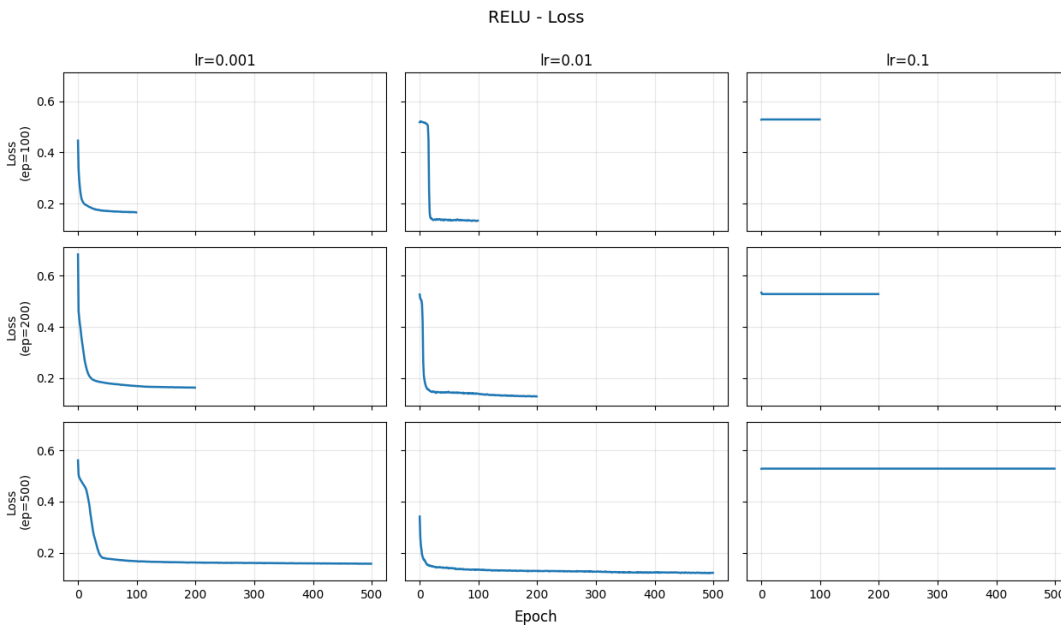
Figure 2: Tanh Testing Accuracy



7.3 ReLU Performance Analysis

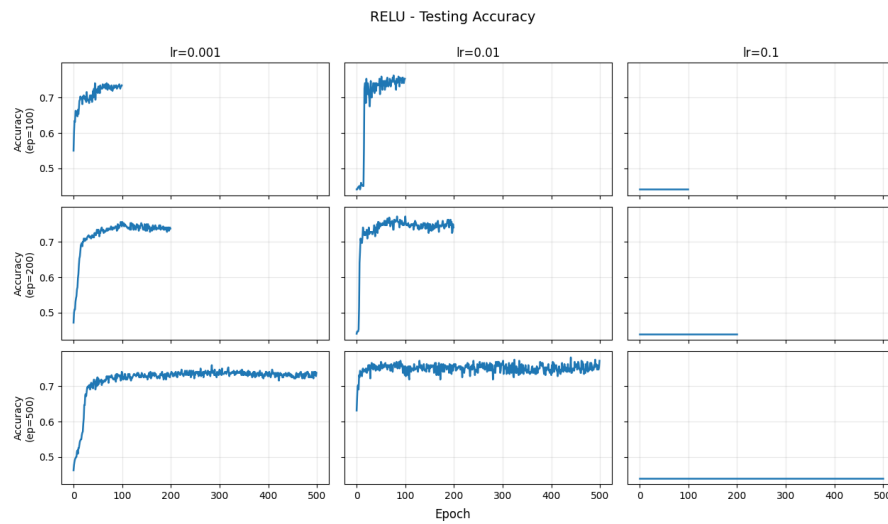
ReLU's performance was very dependent on the learning rate, showing a strong generalization and complete training failure under different conditions. With learning rate 0.01, ReLU achieved its best results, with the testing accuracy improving steadily as training progressed. As shown in Figure 3, the loss decreases rapidly and smoothly, showing efficient gradient flow when neurons remain active. However, at a higher learning rate (0.1), ReLU failed to learn altogether; the loss remains flat and elevated, and the testing's accuracy is low across all epochs. I believe this is due to the exploding gradients and dying neuron problems associated with ReLU under aggressive optimization.

Figure 3: ReLU Loss Curves



This behavior can also be shown in Figure 4, ReLU's testing accuracy graphs. The testing accuracy rises quickly and stabilizes its highest values for this activation function. However, at the 0.1 learning rate, ReLU failed to learn entirely. Overall, ReLU can perform very well when the learning rate is chosen carefully, but small tuning changes can cause the training to become unstable, especially in deep neural networks.

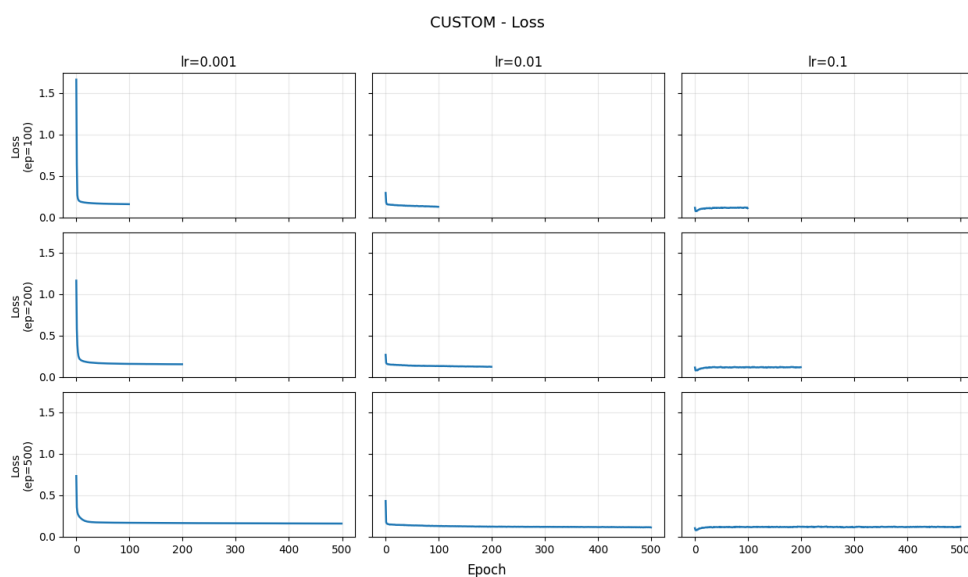
Figure 4: ReLU Testing Accuracy



7.4 Custom Activation Function Analysis

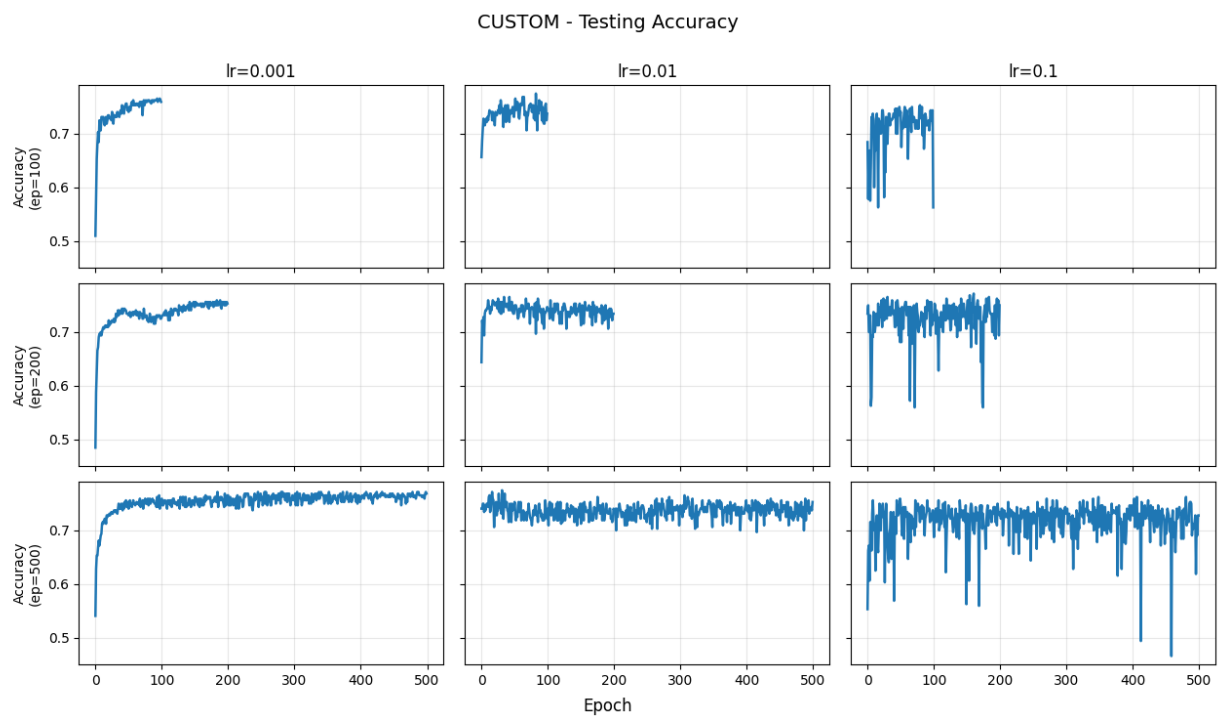
My custom activation function showed the most consistent balance between stability and generalization (for this project). As shown in Figure 5, particularly with the 0.001 learning rate, the loss decreases sharply early in the training and continues to decline gradually without fully flattening, even at higher epoch counts. This shows that learning continues throughout training instead of stopping early, as seen with Tanh.

Figure 5: Custom Loss Curves



The testing accuracy reinforces this observation. As shown in Figure 6, at 500 epochs, the custom activation function achieves its highest testing accuracy, surpassing both Tanh and ReLU. The testing accuracy stabilizes with minimal fluctuation, indicating that longer training improves performance on unseen data rather than leading to overfitting. While training accuracy does increase over time, it remains lower than ReLU's peak training performance, suggesting that the custom function prioritizes generalization over memorization. Including the linear term prevents gradient saturation while maintaining zero-centered outputs, resulting in stable learning and improved final performance.

6: Custom Testing Accuracy



8. Conclusion

This experiment investigated whether a custom activation function could improve gradient flow, training stability, and generalization compared to Tanh and ReLU in a neural network trained on the Wine Quality dataset. All activation functions were evaluated under identical conditions using multiple learning rates and epoch counts. For **this experiment**, the custom activation function achieved the highest testing accuracy and the lowest final loss, showing improved generalization performance relative to both Tanh and ReLU. While ReLU demonstrated strong

training accuracy, it was more sensitive to the learning rate selection and demonstrated complete failure under aggressive optimization. Tanh provided stable learning behavior, but its performance was limited by gradients becoming too small and early plateaus during training.

These results suggest that adding a small linear component to a smooth nonlinear activation function can help maintain gradient flow without sacrificing training stability. However, these findings are specific to this neural network architecture, dataset, and training setup. The results do not imply that the custom activation function is universally better, but instead show that it provides measurable benefits within this controlled experiment. Further work could extend this study by evaluating the custom activation function on different datasets, network architectures, and optimization methods beyond gradient descent, as well as comparing it to other modern activation functions. Nonetheless, this experiment demonstrates that the proposed custom activation function can improve learning stability and generalization compared to Tanh and ReLU within this specific network architecture and dataset.

References

- [1] Alzubaidi, L., Zhang, J., Humaidi, A.J. et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. J Big Data 8, 53 (2021).
<https://doi.org/10.1186/s40537-021-00444-8>
- [2] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Wine Quality [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C56S3T>.
- [3] Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022). Activation functions in deep learning: A comprehensive survey and benchmark. Neurocomputing, 503, 92–108.
<https://doi.org/10.1016/j.neucom.2022.06.111>
- [4] Du, Larry. “All the Activation Functions.” DuBlog, 8 July 2024, dublog.net/blog/all-the-activations/
- [5] Grus, Joel. Data Science From Scratch, 2nd Edition. O'Reilly Media, Inc, 2019.
- [6] Yang, L., Song, Q., Fan, Z., Liu, C., & Hu, M. (2023). Rethinking the activation function in a lightweight network. Multimedia Tools and Applications, 82(1), 1355–1371.
<https://doi.org/10.1007/s11042-022-13217-z>
- [7] Z. Hu, J. Zhang, and Y. Ge, "Handling Vanishing Gradient Problem Using Artificial Derivative," in IEEE Access, vol. 9, pp. 22371-22377, 2021, doi: 10.1109/ACCESS.2021.3054915.